

DBMS Lab: SQL Joins and Transactions

Part A: Create Tables

```
CREATE TABLE roles(role_id INT PRIMARY KEY, role_name VARCHAR(50));
CREATE TABLE users(user_id INT PRIMARY KEY,name VARCHAR(50),email
VARCHAR(50),role_id INT);
CREATE TABLE departments(dept_id INT PRIMARY KEY,dept_name
VARCHAR(50));
CREATE TABLE employees(emp_id INT PRIMARY KEY,emp_name
VARCHAR(50),dept_id INT,manager_id INT,salary DECIMAL(10,2));
CREATE TABLE customers(customer_id INT PRIMARY KEY,customer_name
VARCHAR(50),city VARCHAR(50));
CREATE TABLE orders(order_id INT PRIMARY KEY,customer_id INT,order_date
DATE,total_amount DECIMAL(10,2));
CREATE TABLE accounts(account_no INT PRIMARY KEY,holder_name
VARCHAR(50),balance DECIMAL(10,2));
```

Q1. Display employee names with their department names using INNER JOIN.

```
SELECT emp_name,dept_name FROM employees e INNER JOIN departments d ON
e.dept_id=d.dept_id;
```

Q2. Display all users with their assigned roles using LEFT JOIN.

```
SELECT name,email,role_name FROM users u LEFT JOIN roles r ON
u.role_id=r.role_id;
```

Q3. Display all roles even if no users are assigned using RIGHT JOIN.

```
SELECT name,role_name FROM users u RIGHT JOIN roles r ON u.role_id=r.role_id;
```

Q4. Simulate FULL OUTER JOIN in MySQL.

```
SELECT name,email,role_name FROM users u LEFT JOIN roles r ON
u.role_id=r.role_id UNION SELECT name,email,role_name FROM users u RIGHT
JOIN roles r ON u.role_id=r.role_id;
```

Q5. Display employee and manager names using SELF JOIN.

```
SELECT e.emp_name Employee,m.emp_name Manager FROM employees e LEFT
JOIN employees m ON e.manager_id=m.emp_id;
```

Q6. Display Order ID, Customer Name, City and Total Amount.

```
SELECT o.order_id,customer_name,city,total_amount FROM customers c JOIN orders o
ON c.customer_id=o.customer_id;
```

Q7. Find department-wise employee count, average salary and maximum salary.

```
SELECT dept_name,COUNT(*),AVG(salary),MAX(salary) FROM employees e JOIN  
departments d ON e.dept_id=d.dept_id GROUP BY dept_name;
```

Q8. Display employees earning >50000 in CSE.

```
SELECT emp_name,salary,dept_name FROM employees e JOIN departments d ON  
e.dept_id=d.dept_id WHERE salary>50000 AND dept_name='CSE';
```

Q9. Display customer-wise total purchase in descending order.

```
SELECT customer_name,SUM(total_amount) TotalPurchase FROM customers c JOIN  
orders o ON c.customer_id=o.customer_id GROUP BY customer_name ORDER BY  
TotalPurchase DESC;
```

Q10. Create Students, Courses, Faculty, Enrollments and display Student, Course, Faculty using joins.

```
SELECT student_name,course_name,faculty_name FROM students s JOIN enrollments  
e ON s.student_id=e.student_id JOIN courses c ON e.course_id=c.course_id JOIN  
faculty f ON c.faculty_id=f.faculty_id;
```

Part B: Transactions

Q11. Insert a new account and COMMIT.

```
START TRANSACTION; INSERT INTO accounts VALUES(104,'Anil',25000);  
COMMIT;
```

Q12. Insert a record and ROLLBACK.

```
START TRANSACTION; INSERT INTO accounts VALUES(105,'Ravi',10000);  
ROLLBACK;
```

Q13. Transfer ₹5000 from Account 101 to 102.

```
START TRANSACTION; UPDATE accounts SET balance=balance-5000 WHERE  
account_no=101; UPDATE accounts SET balance=balance+5000 WHERE  
account_no=102; COMMIT;
```

Q14. Simulate failed transaction and rollback.

```
START TRANSACTION; UPDATE accounts SET balance=balance-3000 WHERE  
account_no=101; UPDATE accounts SET balance=balance+3000 WHERE  
account_no=999; ROLLBACK;
```

Q15. Demonstrate SAVEPOINT.

```
START TRANSACTION; UPDATE accounts SET balance=balance+2000 WHERE  
account_no=101; SAVEPOINT s1; UPDATE accounts SET balance=balance-1000  
WHERE account_no=101; ROLLBACK TO s1; COMMIT;
```